

Online supplement:
A Comparative Analysis of Relational Database
Access-Layer
Performance in Express.js for PostgreSQL and
MySQL

Mateusz Miotk

Faculty of Mathematics, Physics and Informatics, University of Gdańsk
`mateusz.miotk@ug.edu.pl`

This supplement carries measurement detail referenced from the main text. All tables are generated by the replication package (<https://github.com/mmioTk/express-db-access-performance>; archived at <https://doi.org/10.5281/zenodo.21339518>) from the same raw data as the main-text tables.

1 Dispersion on MySQL

The main text reports the coefficient-of-variation table for PostgreSQL and the per-engine maxima. Table 1 gives the full per-layer, per-pattern CV on MySQL across the 25 independent replicates (maximum 6.6%, `knex`/insert).

Table 1: Coefficient of variation (%) of throughput across the 25 independent replicates, per access layer and pattern on MySQL. Low values indicate stable measurements.

Layer	<code>point_read</code>	<code>range_scan</code>	<code>deep_fetch</code>	<code>aggregation</code>	<code>write</code>
<code>mysql2</code>	3.2	2.7	2.2	1.8	6.6
<code>knex</code>	2.4	3.0	2.2	2.6	6.6
<code>drizzle</code>	3.0	2.2	2.4	2.2	5.2
<code>prisma</code>	4.2	2.0	1.0	4.0	5.7
<code>sequelize</code>	2.2	2.0	1.9	2.1	5.9
<code>typeorm</code>	2.5	1.9	3.0	2.5	6.0
<code>objection</code>	3.3	3.4	2.5	2.5	5.9
<code>mikroorm</code>	3.5	2.9	3.0	1.9	4.9

2 Idiomatic round-trip counts

Table 2 reports, for each layer, the number of SQL statements the idiomatic deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text’s observation that round-trip count alone does not determine throughput: the two layers issuing four statements (Prisma, Objection) sit at opposite ends of the throughput ladder.

Table 2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs’ eager-loading strategies choose their own counts. Full generated SQL and EXPLAIN plans are in the replication package.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
pg	1	1	2	1	1
knex	1	1	2	1	1
drizzle	1	1	2	1	1
prisma	1	1	4	1	1
sequelize	1	1	1	1	1
typeorm	1	1	2	1	2
objection	1	1	4	1	1
mikroorm	1	1	1	1	1

3 Write throughput under default versus relaxed durability

Table 3 gives the per-layer insert throughput under each engine’s default durability configuration (the paper’s primary write regime) and under the symmetric relaxed regime (secondary, 10 replicates), with the relaxed÷default ratio quoted in the main text.

4 Equal-CPU-budget control

Table 4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores); all nine values are quoted in the main text.

5 Coordinated-omission-corrected open-loop sweep

Table 5 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main

Table 3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 10), with the relaxed÷default ratio. At 50 concurrent connections, group commit amortizes the per-commit flushes: relaxing durability buys at most 14% on PostgreSQL (and $\leq 4\%$ on the layer-bound ORMs) and 19–24% on MySQL.

Layer	PostgreSQL			MySQL		
	default	relaxed	ratio	default	relaxed	ratio
pg	6064	6895	1.14	–	–	–
mysql2	–	–	–	809	966	1.19
pg-tuned	6280	7150	1.14	–	–	–
mysql2-tuned	–	–	–	808	972	1.20
knex	4693	5212	1.11	795	948	1.19
drizzle	4230	4675	1.11	796	950	1.19
prisma	5465	5675	1.04	714	879	1.23
sequelize	2668	2653	0.99	772	925	1.20
typeorm	2595	2675	1.03	732	886	1.21
objection	3926	4245	1.08	787	944	1.20
mikroorm	1571	1607	1.02	663	822	1.24

Table 4: Deep-fetch throughput (req/s, PostgreSQL, median of 3) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). `pg` reaches full throughput on a single core; Prisma requires roughly four to approach it, quantifying the CPU subsidy behind its near-native throughput.

Layer	1 core	2 cores	4 cores
pg	3663	3277	3700
prisma	896	1726	3025
mikroorm	463	577	542

text: achieved throughput, corrected latency percentiles measured from each request’s *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

Table 5: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep/nested fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request’s intended start time and timed-out requests included at their clipped value. [†]marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

Layer	250/s	500/s	1000/s	2000/s	4000/s
pg	2	2	2	2	1201 [†]
prisma	3	2	2	4	2110 [†]
knex	2	2	2	33	9420 [†]
sequelize	2	3	41	10757 [†]	10944 [†]
mikroorm	3	15	11220 [†]	11399 [†]	11878 [†]